

REGIONALNI CENTAR ZA PREDINKUBACIJU U PAMETNOJ INDUSTRIJI



Sufinancira
Europska unija



Projekt je financiran putem ITU
mehanizma u okviru ITP-a



CLOUD & CYBERSECURITY IN INDUSTRIAL SYSTEMS

Ivan Magdić, DevOps Engineer
Varaždin, April 28th, 2026

AGENDA

Cloud

- Cloud in reality
- Cloud-native
- Showcases
- Real-world examples and problems
- Q&A

Experience sharing

- Locally on laptop?
- FOI servers or some other hosting providers (ex. Hetzner, DigitalOcean)?
- Cloud providers like Azure, AWS, Google?



HETZNER

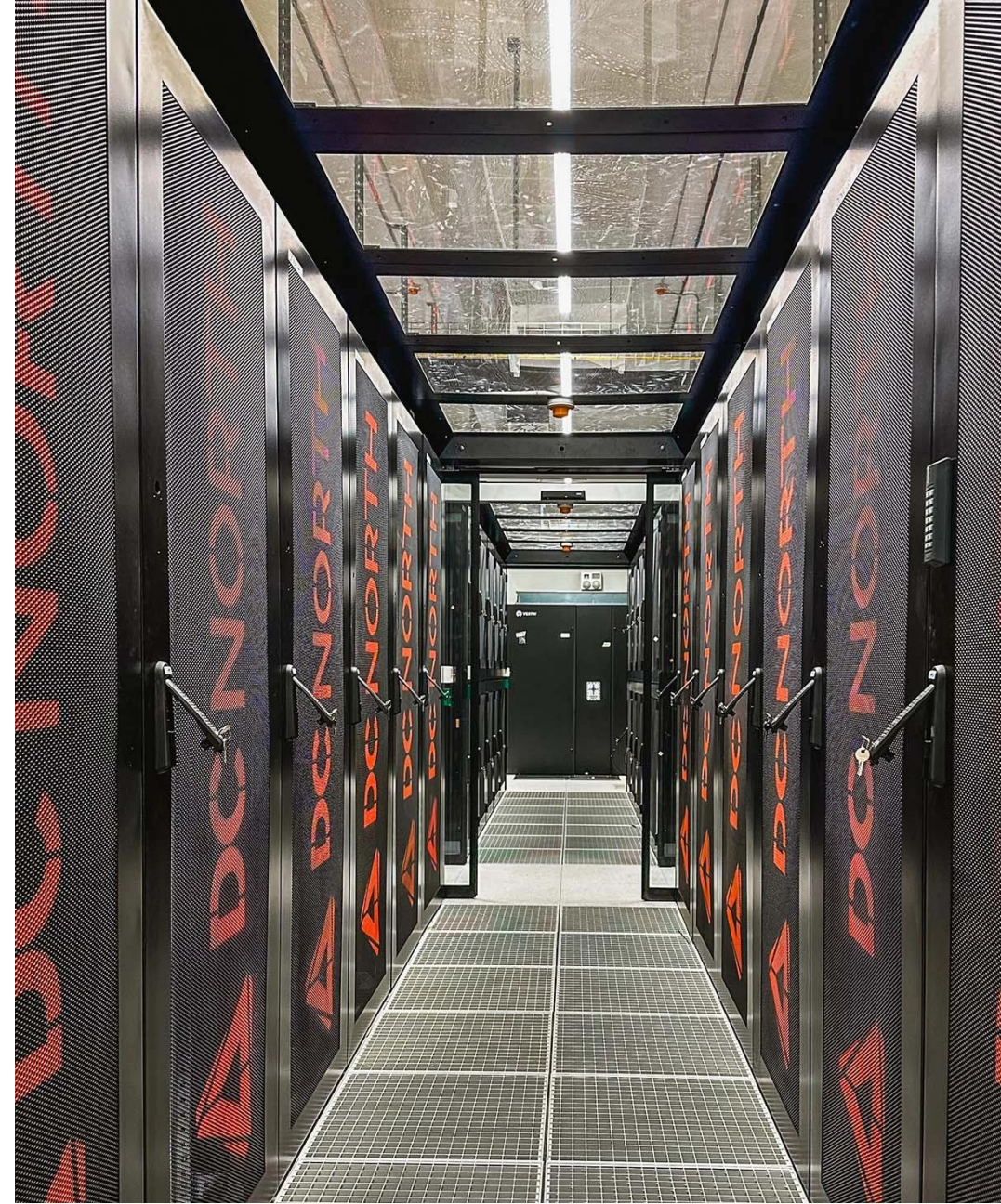


Traditional hosting

- Buy a server
- Install and patch OS
- Deploy application stack
- Maintain everything

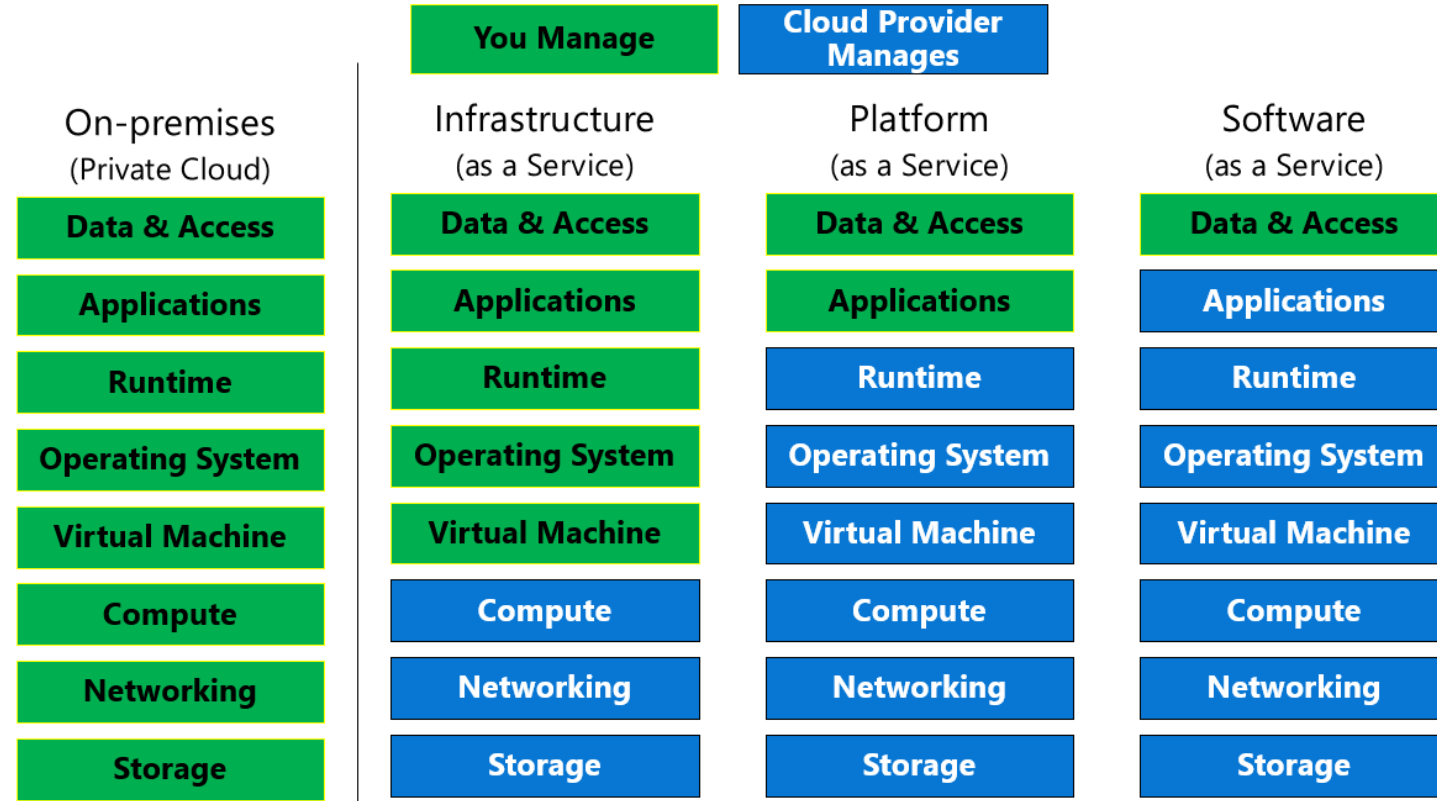
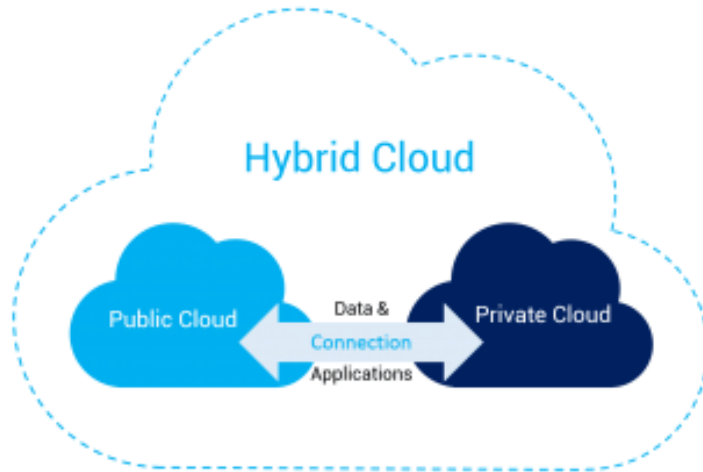
Cons:

- Single point of failure
- Expensive
- Hard to scale
 - everything runs on one machine
 - „just buy a bigger server”



Cloud model

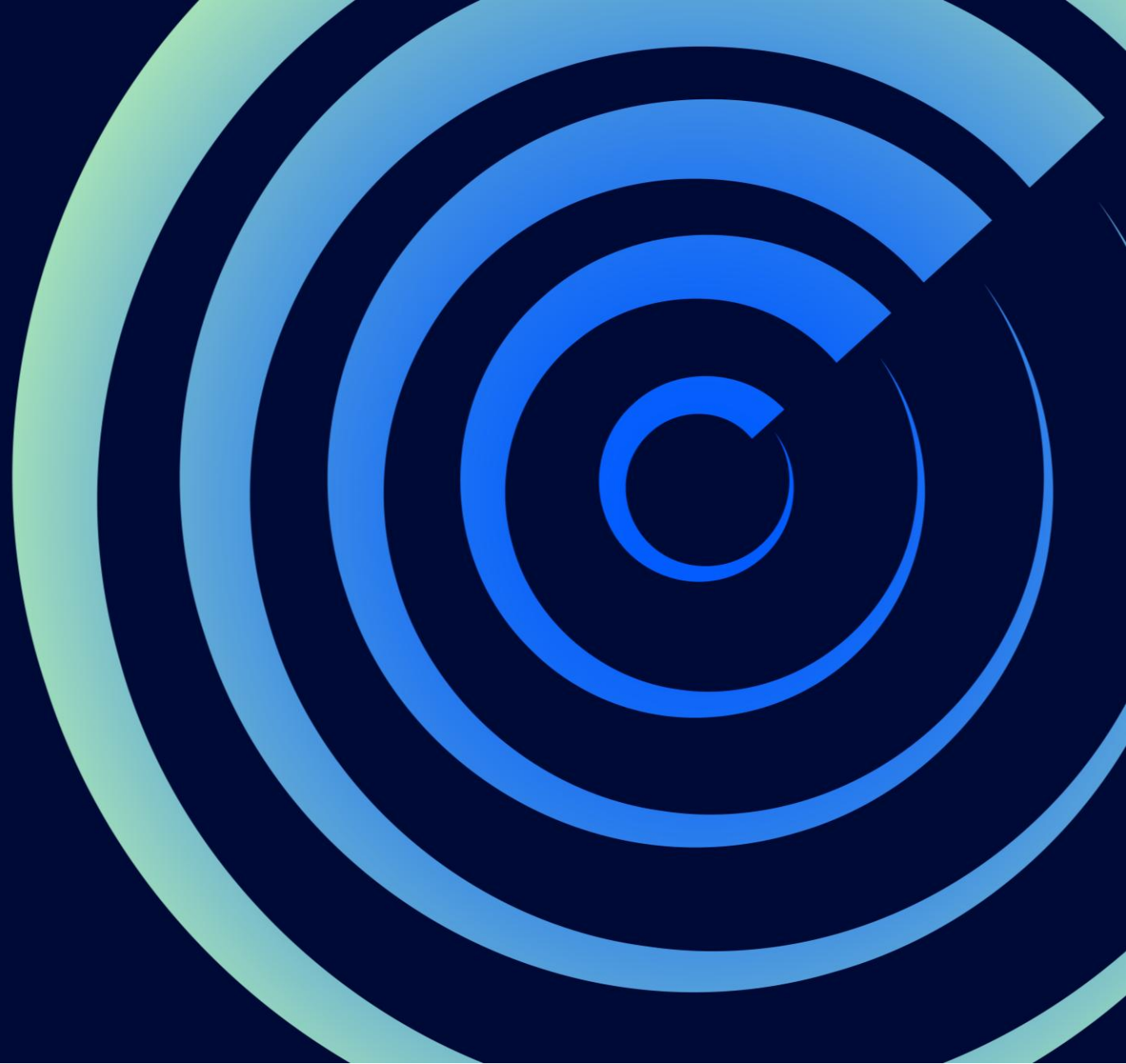
- On-demand self-service
- Broad network access
- Rent infrastructure
- Pay-as-you-go
- API-driven



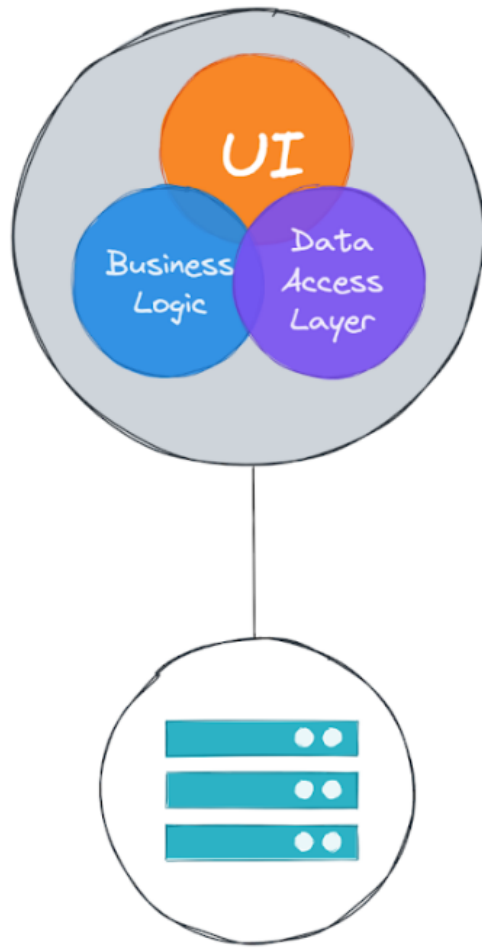
CLOUD-NATIVE

Cloud native practices empower organizations to develop, build, and deploy workloads in computing environments (public, private, hybrid cloud) to meet their organizational needs at scale in a programmatic and repeatable manner.

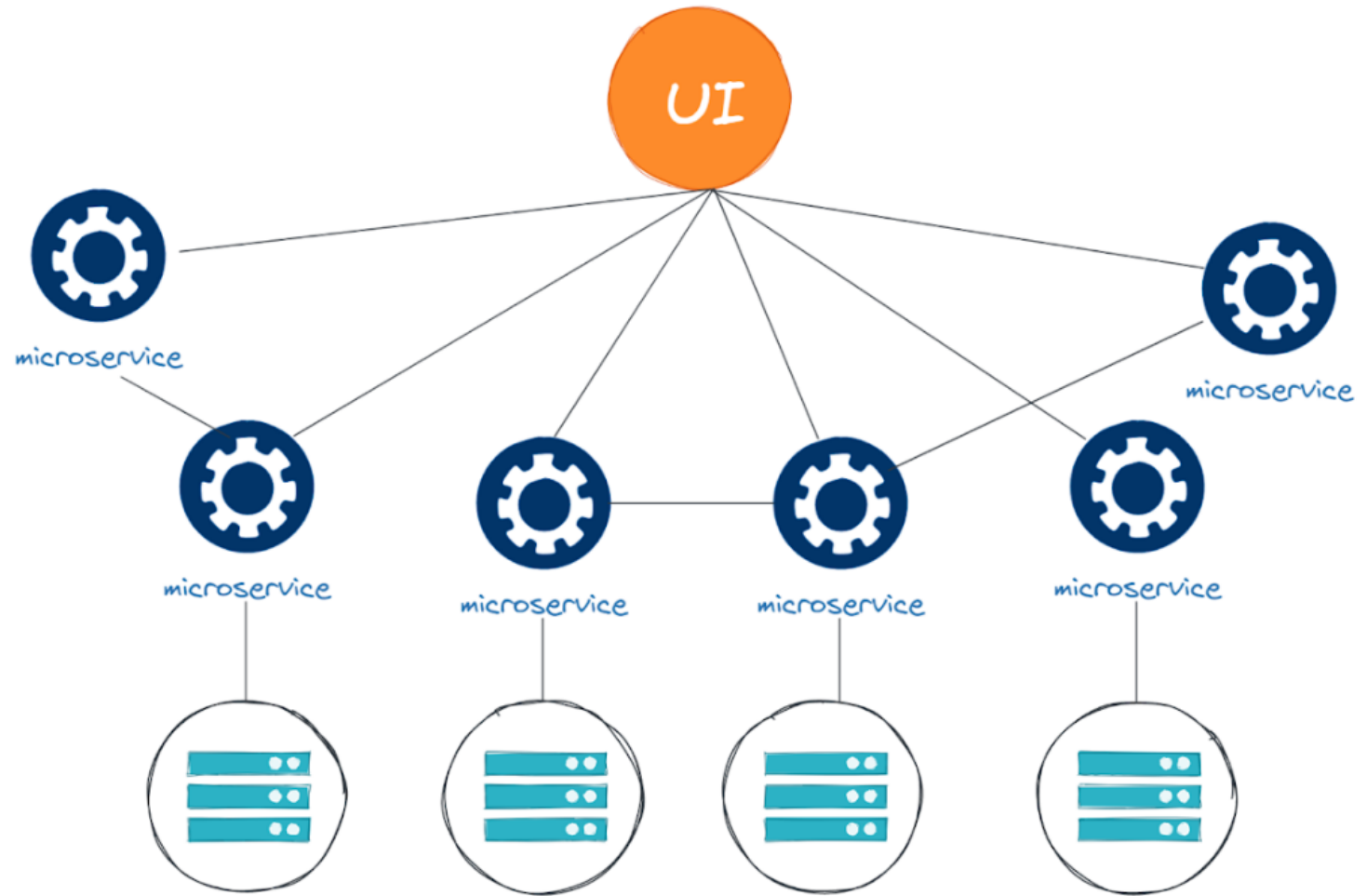
It is **characterized by loosely coupled systems that interoperate in a manner that is secure, resilient, manageable, sustainable, and observable.**



Monolith vs Microservices



Monolithic Architecture



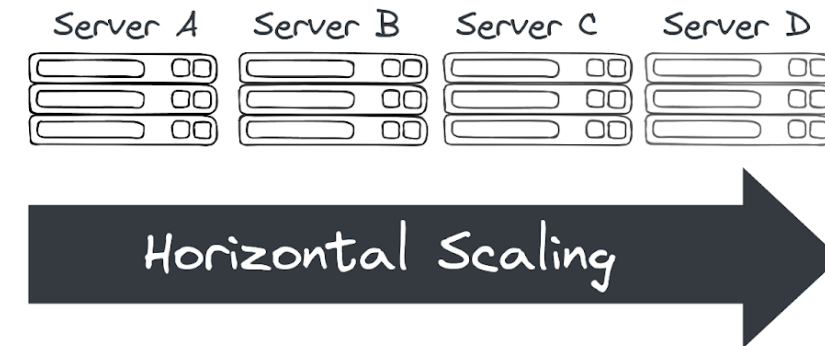
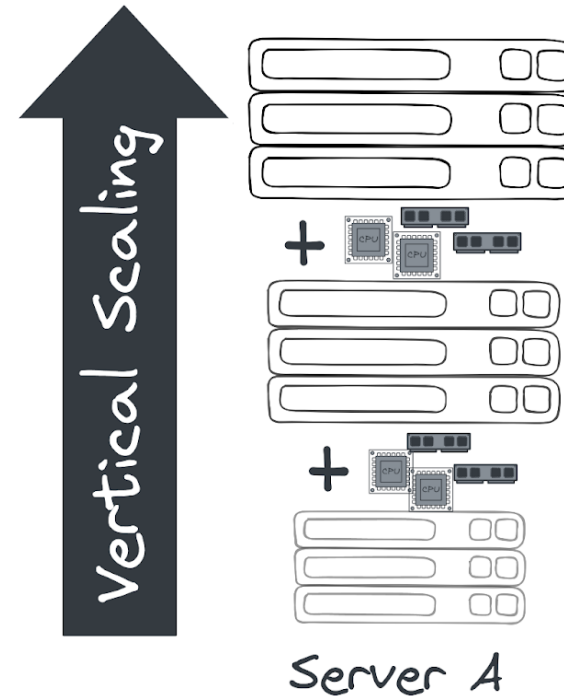
Microservice Architecture

Cloud Native Architecture Characteristics

- **High level of automation**
 - Continuous Integration/Continuous Delivery (CI/CD) pipelines
- **Scalable**
 - Horizontal and vertical scaling
- **Self-healing**
 - Health checks for monitoring
- **(Cost-)efficient**
 - Scale up during high-traffic
 - Scale down to save cost when traffic is low
- **Secure by default**
 - <https://12factor.net> = set of guidelines for developing cloud native applications (**Homework**)

Autoscaling

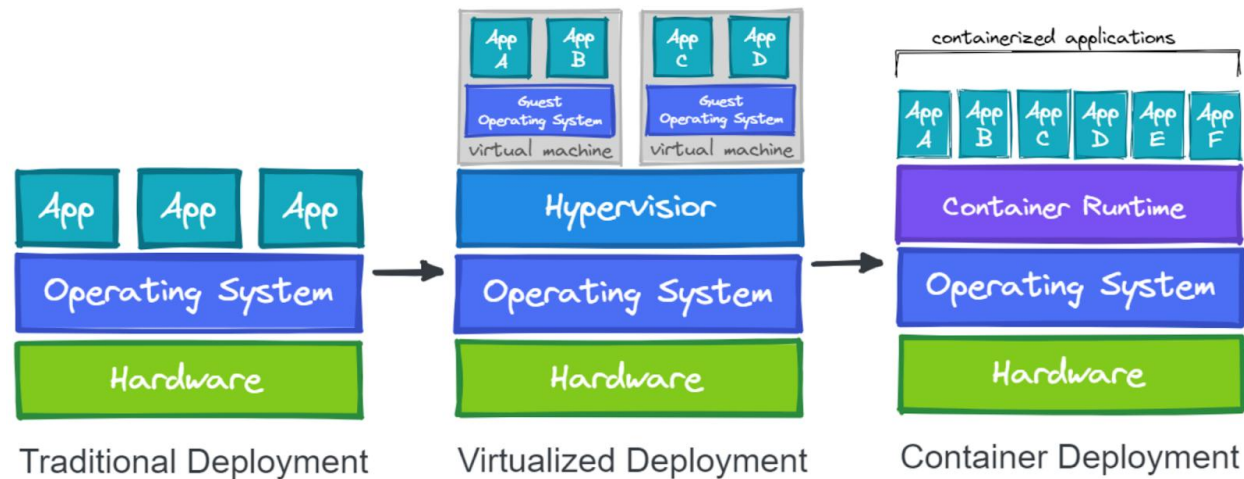
- **Vertical** scaling adds more resources (CPU, memory, storage) to a single server to increase its capacity
- **Horizontal** scaling adds multiple servers (Server A, B, C, D) to distribute workload and improve performance across systems
- *Serverless* – Function as a service
 - The cloud provider abstracts the underlying infrastructure, so developers can deploy software by uploading their code, for example, as .zip files or container images.



CONTAINER ORCHESTRATION

Use of Containers

- **Virtualization** was the most effective method for running applications in isolated, manageable environments. However, because each VM runs its own operating system and kernel, it introduces **resource overhead** when scaling to **many instances**.
- **Containers** solve these issues by packaging applications with their dependencies while sharing the host system's kernel. This makes them **lighter, faster, and more efficient** than running multiple full virtual machines.



Open standards

- A common problem is how to build and **distribute** software packages, since applications have many requirements and dependencies on the **underlying operating system** and the **application runtime**. Containers evolved as a standardized way to package and ship modern software
- Docker – container technology (synonymous for **containerization**)
- [Open Container Initiative \(OCI\)](#)
 - [Image-spec](#)
 - [Runtime-spec](#)
- [Container Network Interface \(CNI\)](#)
- [Container Storage Interface \(CSI\)](#)
- [Service Mesh Interface \(SMI\)](#)

CONTAINER ORCHESTRATION

Use of Containers

- Biggest problem that containers solve are „it works on my machine”
- How would you deploy Python web app without containers?
 - Install and configure the basic operating system
 - Install the core Python packages to run the program
 - Install Python extensions that your program uses
 - Configure networking for your system
 - Connect to third-party systems like a database, cache, or storage
- = All steps are error prone

CONTAINER ORCHESTRATION

Structure of a Container Image

OCI images can be built by reading the instructions in a **Dockerfile**. The instructions are almost identical to those for installing an application on a server

We can build the image with the following command if we have Docker installed:

- **docker build -t my-python-image -f Dockerfile**

To distribute these images, you can use a container registry. This is nothing more than a web server for uploading and downloading images

- **docker push my-registry.com/my-python-image**
- **docker pull my-registry.com/my-python-image**

```
1  # Every container image starts with a base image.
2  # This could be your favorite linux distribution
3  FROM ubuntu:24.04
4
5  # Run commands to add software and libraries to your image
6  # Here we install python3 and the pip package manager
7  RUN apt-get update && \
8      apt-get -y install python3 python3-pip
9
10 # The copy command can be used to copy your code to the image
11 # Here we copy a script called "my-app.py" to the containers filesystem
12 COPY my-app.py /app/
13
14 # Defines the workdir in which the application runs
15 # From this point on everything will be executed in /app
16 WORKDIR /app
17
18 # The process that should be started when the container runs
19 # In this case we start our python app "my-app.py"
20 CMD ["python3", "my-app.py"]
```

CONTAINER ORCHESTRATION

Container Orchestration Fundamentals

- When managing and deploying many containers, it quickly becomes clear that you need a dedicated system to **orchestrate** and **manage** them efficiently.
- Container orchestration systems allow you to create a cluster of **multiple servers** and run **containers** across them efficiently.
- These systems typically have two main components:
 - **control plane**, which manages the cluster and container operations
 - **worker nodes**, which run the containers
- **Kubernetes** has emerged as the industry standard for container orchestration, offering powerful, flexible, and scalable management for containerized applications



KUBERNETES

Architecture

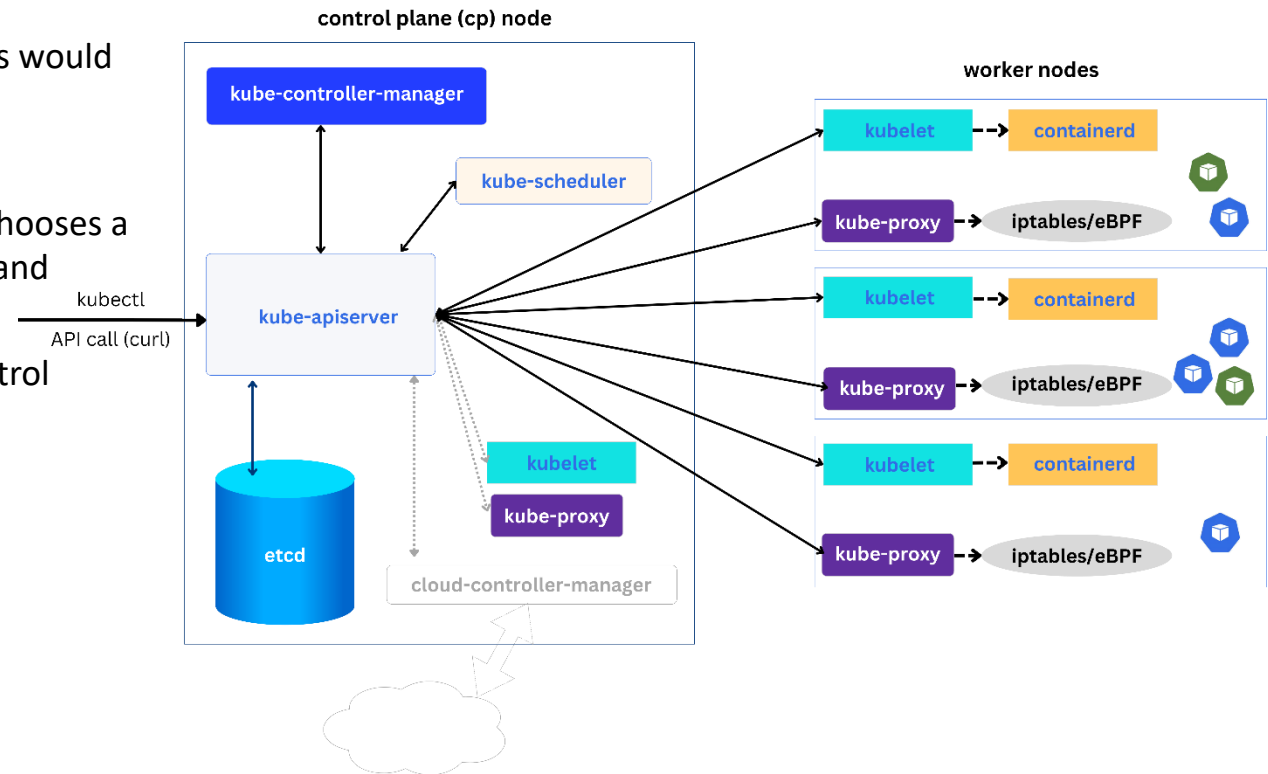
- Kubernetes is typically deployed as a **cluster**, meaning it runs across multiple servers that share workloads, improve reliability, and scale applications efficiently
- Control plane node(s)
 - nodes act as the “brains” of the cluster
 - run components that manage the overall system, including workload scheduling, deployments, and self-healing behaviors
- Worker nodes
 - responsible for running applications - nothing more. They do not make decisions on their own; instead, they follow instructions from the control plane, such as when to start or stop containers



KUBERNETES

Architecture

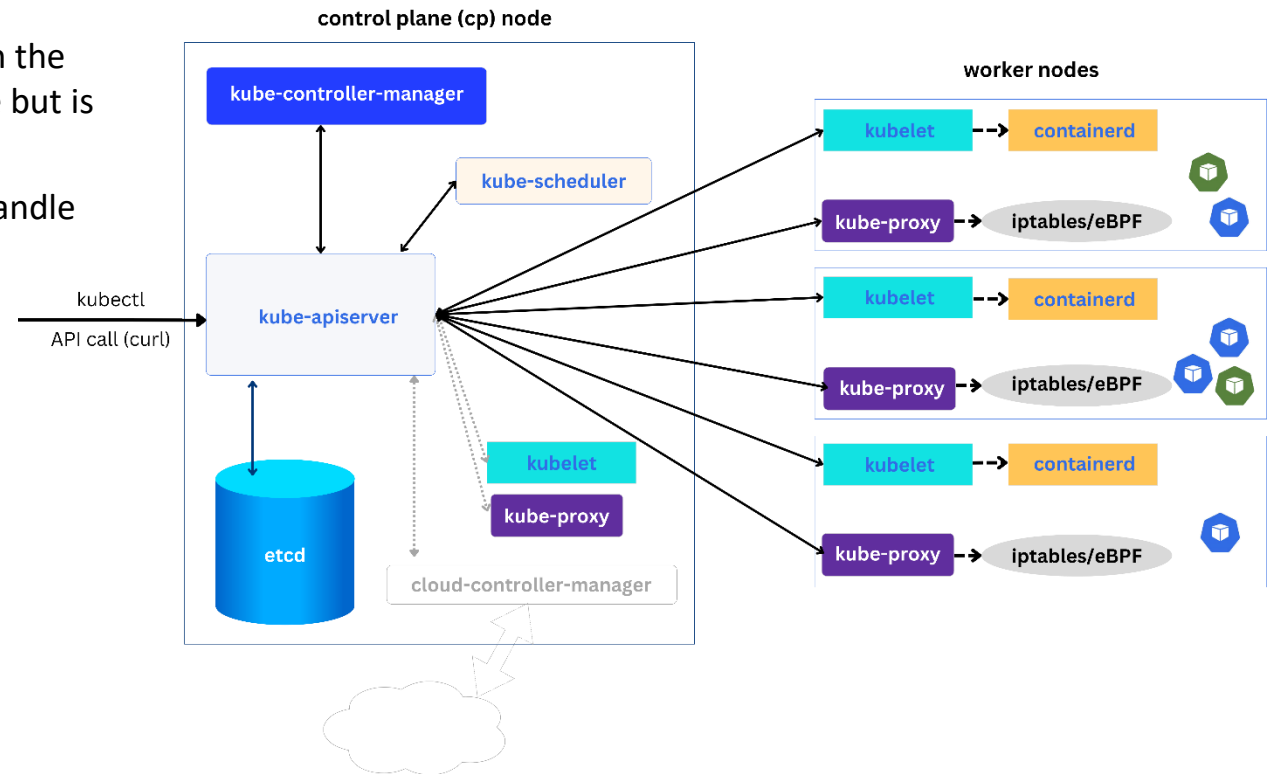
- Like a microservice architecture you would choose for your own application, Kubernetes incorporates multiple smaller services that need to be installed on the nodes.
- All components interact with the **api-server** and this is where users would access the cluster
- A database which holds the state of the cluster - **etcd**
- When a new workload should be scheduled, the **kube-scheduler** chooses a worker node that could fit, based on different properties like CPU and memory
- **kube-controller-manager** - contains different non-terminating control loops that manage the state of the cluster



KUBERNETES

Architecture

- Like a microservice architecture you would choose for your own application, Kubernetes incorporates multiple smaller services that need to be installed on the nodes.
- The **container runtime** is responsible for running the containers on the worker node. For a long time, **Docker** was the most popular choice but is now replaced in favor of other runtimes like **containerd**
- The **kubelet** talks to the api-server and the container runtime to handle the final stage of starting containers (runs on every worker node)
- **kube-proxy** is a network proxy that handles inside and outside communication of your cluster



KUBERNETES

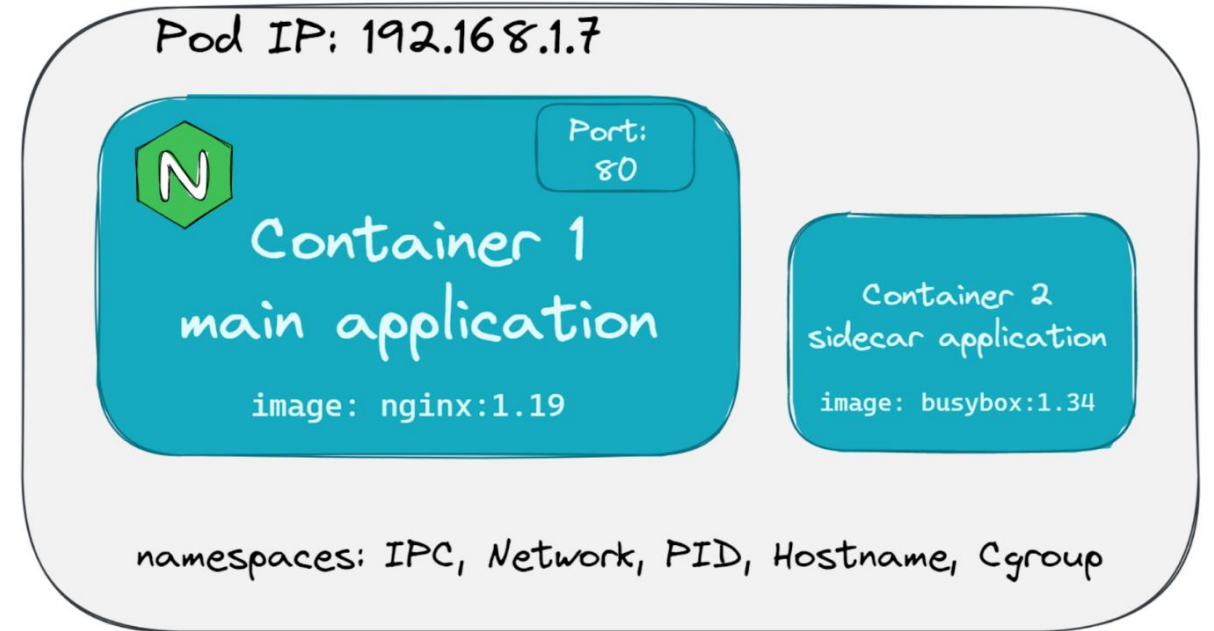
Objects

- One of Kubernetes core principles is that it represents system behavior through a collection of abstract resources (objects) that define how workloads should run and be managed
- A workload is an application running on Kubernetes. Whether your workload is a single component or several that work together, on Kubernetes you run it inside a set of **pods**. In Kubernetes, a Pod represents a set of one or more running **containers** on your cluster
- Users define these objects using **YAML** and submit them to the Kubernetes API server, where the definitions are **validated** and then used to **create** or **modify** resources in the cluster

KUBERNETES

Pods

- Because a **pod** is the smallest deployable resource in Kubernetes, the platform does not interact with containers directly - it always operates through pods
- The pod abstraction was created to support scenarios where multiple tightly coupled processes need to run together. All containers within a pod share the **same IP address** and can communicate over **localhost**, and they can also share storage volumes through a common filesystem



KUBERNETES

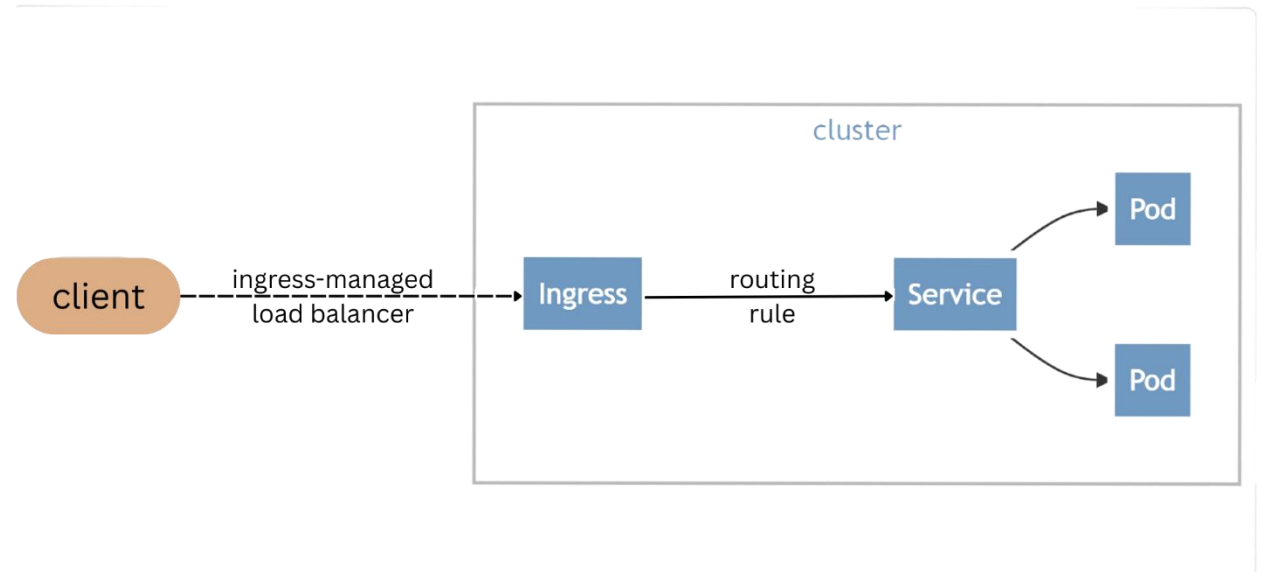
Objects

- **Deployment**
 - A Deployment is one of the most powerful and commonly used controllers in Kubernetes. It manages the full lifecycle of stateless applications, providing declarative updates to Pods and ReplicaSets
- **StatefulSet**
 - A StatefulSet is designed for managing stateful applications that require stable identities, persistent storage, and predictable Pod ordering. Unlike Deployments, which treat Pods as interchangeable, StatefulSets assign each Pod a unique, persistent identifier
- **DaemonSet**
 - A DaemonSet ensures that a specific Pod runs on every node (or on a subset of nodes labeled with a particular label) in a Kubernetes cluster. As new nodes are added, the DaemonSet automatically schedules the required Pods onto them; when nodes are removed, the Pods are cleaned up
- Networking type: **Service**
 - exposes one or more Pods as a network endpoint

KUBERNETES

Ingress and exposing the applications to outside world

- **Ingress**
 - When you need more flexible or HTTP-aware routing, you can use an **Ingress** resource. Ingress exposes HTTP and HTTPS endpoints from outside the cluster to internal **Services** by applying user-defined routing rules. These rules are enforced by an **Ingress Controller**, which implements the actual networking logic
- Typical features of ingress controllers include:
 - LoadBalancing
 - TLS offloading or termination
 - Name-based virtual hosting
 - Path-based routing



KUBERNETES

Objects

In the manifest (YAML file) for the Kubernetes object you want to create, you'll need to set values for the following fields:

- **apiVersion** - Which version of the Kubernetes API you're using to create this object
- **kind** - What kind of object you want to create
- **metadata** - Data that helps uniquely identify the object, including a name string, UID, and optional namespace
- **spec** - What state you desire for the object

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2 # tells deployment to run 2 pods matching the template
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.29
          ports:
            - containerPort: 80
```

KUBERNETES

Interacting with Kubernetes

- To interact with the Kubernetes API, use the official command-line tool, **kubectl**.
 - `kubectl --help`

```
magdiiv@4MXWX54:/mnt/c/Users/magdiiv/Code/aks-nfa-azcni-buds-nfa-prd-weu/application-services$ k api-resources
```

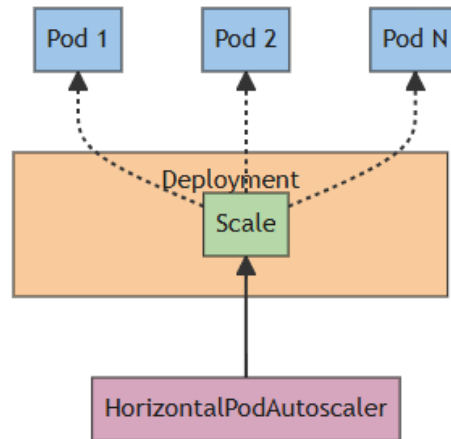
NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
componentstatuses	cs	v1	false	ComponentStatus
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pv	v1	false	PersistentVolume
Pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service
challenges		acme.cert-manager.io/v1	true	Challenge
orders		acme.cert-manager.io/v1	true	Order
mutatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	MutatingWebhookConfiguration
validatingadmissionpolicies		admissionregistration.k8s.io/v1	false	ValidatingAdmissionPolicy
validatingadmissionpolicybindings		admissionregistration.k8s.io/v1	false	ValidatingAdmissionPolicyBinding
validatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	ValidatingWebhookConfiguration
customresourcedefinitions	crd,crds	apiextensions.k8s.io/v1	false	CustomResourceDefinition
apiservices		apiregistration.k8s.io/v1	false	APIService
azureapplicationgatewayrewrites	agrewrite	appgw.ingress.azure.io/v1beta1	true	AzureApplicationGatewayRewrite
controllerrevisions		apps/v1	true	ControllerRevision
daemonsets	ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true	Deployment
replicasets	rs	apps/v1	true	ReplicaSet
statefulsets	sts	apps/v1	true	StatefulSet

- DEMO TIME !

KUBERNETES

Horizontal Pod Autoscaler (HPA)

- The **Horizontal Pod Autoscaler (HPA)** is the most commonly used autoscaler and it monitors metrics such as CPU or memory usage and increases or decreases the number of Pod replicas when thresholds are reached.
- For example, if a Pod is configured to use up to 500MiB of memory with an 80% threshold, Kubernetes will schedule a second Pod once usage exceeds 400MiB. As demand continues to grow, additional Pods are created so that total capacity scales horizontally.



```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: nginx
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind: Deployment
9      name: nginx
10   minReplicas: 1
11   maxReplicas: 10
12   metrics:
13   - type: Resource
14     resource:
15       name: cpu
16       target:
17         type: Utilization
18         averageUtilization: 50
```

KUBERNETES

Vertical Pod Autoscaler (VPA)

- The Vertical Pod Autoscaler (VPA) adjusts a Pod's CPU and memory requests and limits dynamically rather than adding more replicas.
- This is useful for workloads that scale better by increasing resources per Pod rather than adding more Pods. However, vertical scaling is constrained by **the capacity of individual nodes** and becomes **less effective** once limits are reached.
- In order to use autoscaling features in Kubernetes, we need to install and use **metrics-server**

```
1  apiVersion: autoscaling.k8s.io/v1
2  kind: VerticalPodAutoscaler
3  metadata:
4    name: nginx-vpa
5  spec:
6    targetRef:
7      apiVersion: "apps/v1"
8      kind: Deployment
9      name: nginx-deployment
10   updatePolicy:
11     updateMode: "Recreate"
12   resourcePolicy:
13     containerPolicies:
14     - containerName: "nginx"
15       minAllowed:
16         cpu: 100m
17         memory: 128Mi
18       maxAllowed:
19         cpu: 2
20         memory: 2Gi
21       controlledResources:
22       - cpu
23       - memory
24     controlledValues: RequestsAndLimits
```

KUBERNETES

Monitoring - metrics

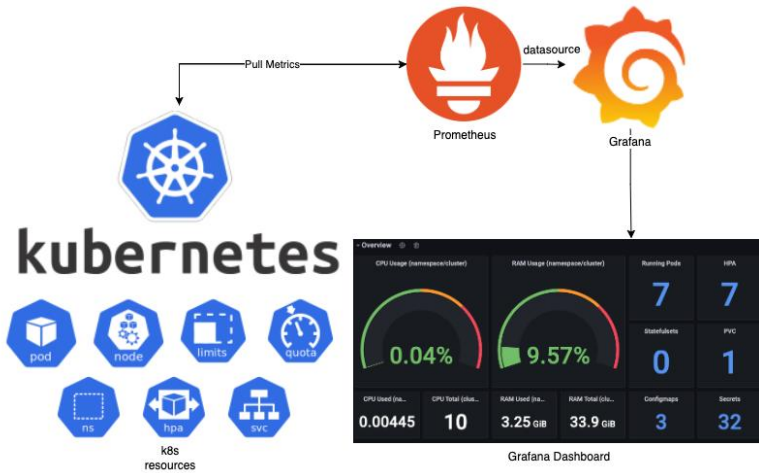
- **Prometheus** collects metrics emitted by applications and servers as time-series data, simple data points that include a **timestamp**, **optional labels**, and a **measured value**
- To make metrics available to Prometheus, applications typically expose them through an **HTTP** endpoint, commonly at **/metrics**

```
1 # HELP queue_length The number of items in the queue.
2 # TYPE queue_length
3 gauge queue_length 42
4 # HELP http_requests_total The total number of handled HTTP requests.
5 # TYPE http_requests_total counter
6 http_requests_total 7734
7 # HELP http_request_duration_seconds A histogram of the HTTP request durations in seconds.
8 # TYPE http_request_duration_seconds histogram http_request_duration_seconds_bucket{le="0.05"} 4599
9 http_request_duration_seconds_sum 88364.234
10 http_request_duration_seconds_count 227420
11 # HELP http_request_duration_seconds A summary of the HTTP request durations in seconds.
12 # TYPE http_request_duration_seconds summary
13 http_request_duration_seconds{quantile="0.5"} 0.052
14 http_request_duration_seconds_sum 88364.234
15 http_request_duration_seconds_count 227420
```

KUBERNETES

Monitoring – visualize metrics

- Monitoring only becomes valuable when the collected data is used effectively. **Grafana** is the most common companion to Prometheus and is widely used to build dashboards that visualize metrics
- Although Grafana supports many data sources, Prometheus is one of the most popular

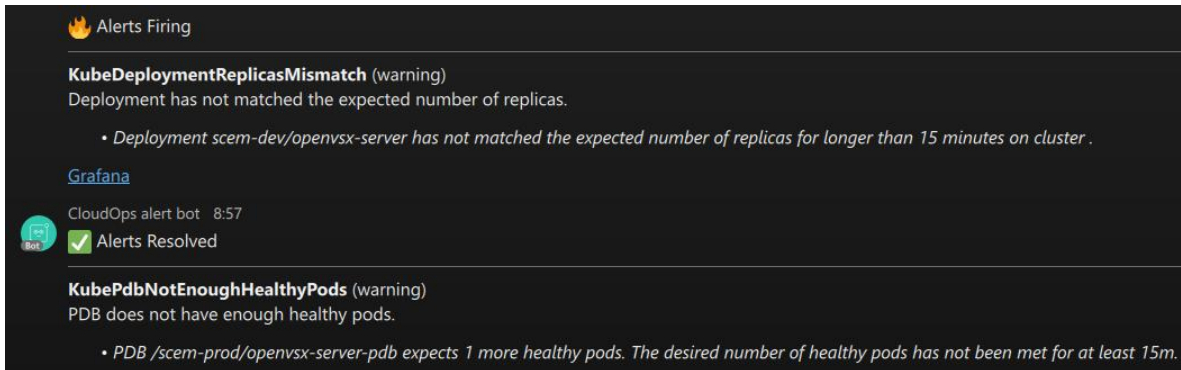


KUBERNETES

Monitoring - alerting

- Another key component of the Prometheus ecosystem is Alertmanager. Prometheus can evaluate alerting rules and fire alerts when metric thresholds are met or exceeded.
- Alertmanager then handles routing these alerts to your preferred destination, such as **email**, **chat** platforms, or **specialized incident-management** tools.

```
1 groups:
2   - name: example
3     rules:
4       - alert: HighRequestLatency
5         expr: job:request_latency_seconds:mean5m{job="myjob"} > 0.5 for: 10m
6         labels:
7           severity: page
8         annotations:
9           summary: High request latency
```



 Alerts Firing

KubeDeploymentReplicasMismatch (warning)
Deployment has not matched the expected number of replicas.

- Deployment *scem-dev/openvsx-server* has not matched the expected number of replicas for longer than 15 minutes on cluster.

[Grafana](#)

CloudOps alert bot 8:57

  Alerts Resolved

KubePdbNotEnoughHealthyPods (warning)
PDB does not have enough healthy pods.

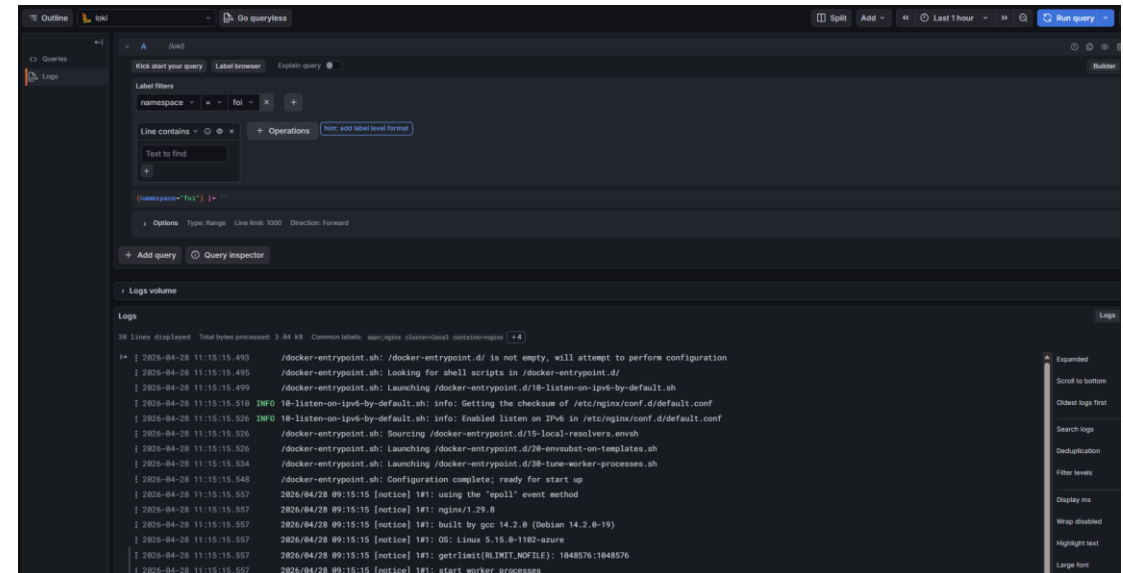
- PDB */scem-prod/openvsx-server-pdb* expects 1 more healthy pods. The desired number of healthy pods has not been met for at least 15m.

KUBERNETES

Monitoring - logging

- Modern programming languages and application frameworks include robust logging tools, making it straightforward to write log messages to files with appropriate severity levels
- Command line tools like **docker** or **kubectl** provide a command to show the logs of containerized processes if you let them log directly to the console

```
magdiiv@4MXW54:/mnt/c/Users/magdiiv/Code/aks-nfa-azcni-buds-nfa-prd-weu/application-services$ k logs nginx-deployment-578c8ff859-9wjgb -n foi
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/28 09:15:15 [notice] 1#1: using the "epoll" event method
2026/04/28 09:15:15 [notice] 1#1: nginx/1.29.8
2026/04/28 09:15:15 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/28 09:15:15 [notice] 1#1: OS: Linux 5.15.0-1102-azure
2026/04/28 09:15:15 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/28 09:15:15 [notice] 1#1: start worker processes
2026/04/28 09:15:15 [notice] 1#1: start worker process 30
2026/04/28 09:15:15 [notice] 1#1: start worker process 31
2026/04/28 09:15:15 [notice] 1#1: start worker process 32
2026/04/28 09:15:15 [notice] 1#1: start worker process 33
```



CONTAINER ORCHESTRATION

Security

- When containers run on a host machine, they all share the **same operating system kernel**. This shared kernel can pose a security risk if containers are granted **excessive permissions** (as the root or administrator user)
- It's essential to verify that publicly available images have not **been tampered** with or contain **malicious software** before using them.

The attacker compromised the `jasonsaayman` npm account, the primary maintainer of the `axios` project. The account's registered email was changed to `ifstap@proton.me` — an attacker-controlled ProtonMail address. Using this access, the attacker published malicious builds across both the `1.x` and `0.x` release branches simultaneously, maximizing the number of projects exposed.

Both `axios@1.14.1` and `axios@0.30.4` are recorded in the npm registry as published by `jasonsaayman`, making them indistinguishable from legitimate releases at a glance. Both versions were published using the compromised npm credentials of a lead `axios` maintainer, bypassing the project's normal GitHub Actions CI/CD pipeline.

Attack Timeline

I don't have an exact timeline for when the initial compromise occurred, but this is the sequence of events for the package itself:

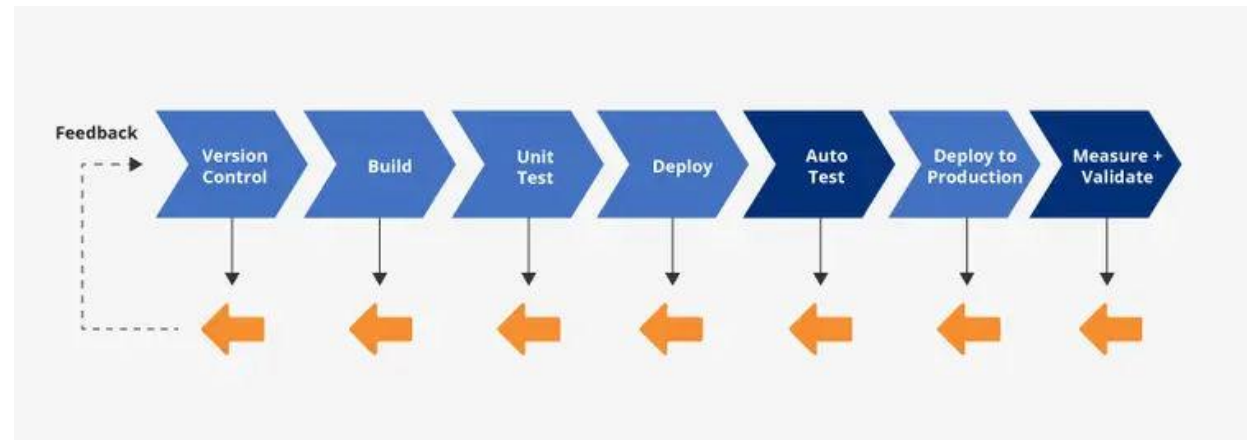
- About 2 weeks before March 31: social engineering campaign initiated against the lead maintainer
- March 30, 05:57 UTC: `plain-crypto-js@4.2.0` published to npm
- March 31, 00:21 UTC: `axios@1.14.1` published with `plain-crypto-js@4.2.1` injected
- March 31, around 01:00 UTC: `axios@0.30.4` published with the same payload
- March 31, around 01:00 UTC: first external detections
- March 31, around 01:00 UTC: community members file issues reporting the compromise. The attacker deletes them using the compromised account.
- March 31, 01:38 UTC: `axios` collaborator DigitalBrainJS opens PR [chore\(ci\): add deprecate action: #10591](#) to deprecate the compromised versions, flags the deleted issues to the community, and contacts npm directly
- March 31, 03:15 UTC: malicious versions removed from npm
- March 31, 03:29 UTC: `plain-crypto-js` removed from npm

- Latest impactful exploitation: <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>

CI/CD

Basics

- With applications becoming smaller and deployments **more frequent**, automating the deployment process became a natural and necessary evolution
 - **Continuous Integration (CI)** focuses on automatically building and testing code whenever changes are made.
 - **Continuous Delivery (CD)** automates the deployment of built artifacts to environments. In cloud-native workflows, deployments typically progress through development and staging environments before being promoted to production



GITOPS

Basics

- **Infrastructure as Code (IaC)** was a breakthrough in how infrastructure is provisioned and managed. By defining environments as code rather than configuring them manually, organizations can deliver infrastructure **faster**, more consistently, and with **fewer errors**
- Just as developers submit pull requests to propose changes to software, **GitOps** applies the same workflow to platform and infrastructure changes. Each change is **reviewed**, **validated**, and **tested** using CI pipelines before being merged, allowing infrastructure updates to follow the same auditing, approval, and collaboration patterns as application code
- **ArgoCD** acts as a Kubernetes controller that continuously reconciles cluster resources with repository configuration
- **Terraform** allows you to define, provision, and manage cloud and on-premise resources using declarative configuration file

